

Apostila 1 — Fundamentos de Python para Economia

1. Por que Python?

Python é a linguagem mais usada em economia por três motivos:

1. **Legibilidade** — código parece quase português
2. **Ecossistema** — pandas, statsmodels, matplotlib são feitos para análise de dados
3. **Comunidade** — qualquer dúvida já tem resposta no Stack Overflow

Economistas do FMI, Banco Mundial, BCB e quase todos os programas de pós-graduação usam Python.

2. Primeiros passos

2.1. O que é um script?

Um arquivo .py é uma lista de instruções que o computador executa de cima para baixo.

```
# isso é um comentário — o computador ignora
# serve para explicar o código

print("Hello, Economia!") # print exibe algo na tela
```

Salve como teste.py e execute:

```
python teste.py
```

Saída:

```
Hello, Economia!
```

2.2. Print — sua ferramenta mais útil

print() exibe qualquer coisa. Use **sempre** que quiser entender o que seu código está fazendo.

```
print("IPCA de junho: 0.16%")
```

3. Variáveis

Variável é um nome que guarda um valor.

```
ipca_junho = 0.16
selic = 10.50
pib_2025 = 3.2

print(ipca_junho) # 0.16
print(selic)      # 10.5
```

Regras do nome de variável: - Pode conter letras, números e underscore _ - Não pode começar com número - Maiúsculas e minúsculas são diferentes (IPCA ≠ ipca) - Use nomes claros: taxa_juros é melhor que tj

4. Tipos de dados

4.1. Números inteiros (int)

```
ano = 2026
populacao = 214_000_000 # underscore ignorado, só pra leitura
```

4.2. Números decimais (float)

```
ipca = 0.16
cambio = 5.45
pib_per_capita = 45000.00
```

4.3. Texto (str)

```
pais = "Brasil"
sigla = 'BR'
mensagem = "O PIB cresceu 3.2% em 2025"
```

4.4. Booleano (bool)

Verdadeiro ou falso:

```
em_recessao = False
crescendo = True
```

4.5. Verificando o tipo

```
print(type(ipca))          # <class 'float'>
print(type(pais))          # <class 'str'>
print(type(em_recessao))   # <class 'bool'>
```

5. Operadores

5.1. Aritméticos

```
soma = 5 + 3          # 8
subtracao = 5 - 3     # 2
multiplicacao = 5 * 3 # 15
divisao = 5 / 3       # 1.666...
divisao_inteira = 5 // 3 # 1
resto = 5 % 3         # 2 (resto da divisão)
potencia = 5 ** 3     # 125

# Aplicação econômica:
valor_futuro = 1000 * (1 + 0.10) ** 5 # 1610.51 (juros compostos)
print(f"R$ {valor_futuro:.2f}")
```

5.2. Comparação (resultado é bool)

```
10 > 5    # True
10 < 5    # False
10 == 10  # True (igualdade)
10 != 5   # True (diferente)
10 >= 10  # True
5 <= 3    # False
```

5.3. Lógicos

```
True and False # False (os dois precisam ser True)
True or False  # True (pelo menos um True)
not True       # False (inverte)
```

Aplicação econômica:

```
inflacao = 0.60
desemprego = 8.5

if inflacao > 0.5 and desemprego > 8.0:
    print("Estagflação?")
```

6. Strings em detalhe

```
# Concatenação
pais = "Brasil"
ano = 2026
frase = pais + " em " + str(ano) # "Brasil em 2026"
# str() converte número pra texto

# F-string (MUITO mais prático)
frase2 = f"{pais} em {ano}"
print(frase2) # "Brasil em 2026"

# Formatação de números
ipca = 0.1667
print(f"IPCA: {ipca:.2f}%") # "IPCA: 0.17%" (2 casas decimais)
print(f"IPCA: {ipca:.1f}%") # "IPCA: 0.2%" (1 casa decimal)

# Métodos úteis
texto = " brasil "
texto.upper() # " BRASIL "
texto.lower() # " brasil "
texto.strip() # "brasil" (remove espaços)
texto.replace("a", "o") # "brosil"
texto.split("r") # [" b", "asil "]
```

7. Listas

Lista é uma coleção ordenada de valores.

```
# Criando listas
inflacoes = [0.50, 0.70, 0.88, 0.67, 0.58, 0.16]
países = ["Brasil", "Argentina", "Chile", "Colômbia"]
mista = [1, "texto", True, 3.14] # tipos diferentes (possível, mas evite)

# Acessando elementos (índice começa em 0!)
inflacoes[0] # 0.50 (primeiro)
inflacoes[1] # 0.70 (segundo)
inflacoes[-1] # 0.16 (último)
inflacoes[-2] # 0.58 (penúltimo)

# Fatiamento (slicing) – [início:fim:passo]
inflacoes[0:3] # [0.50, 0.70, 0.88] (índices 0,1,2)
inflacoes[:3] # [0.50, 0.70, 0.88] (mesma coisa)
inflacoes[3:] # [0.67, 0.58, 0.16] (do índice 3 até o fim)
inflacoes[::2] # [0.50, 0.88, 0.58] (pulando de 2 em 2)
inflacoes[::-1] # [0.16, 0.58, ...] (inverte a ordem)

# Métodos de lista
numeros = [1, 2, 3]
numeros.append(4) # [1, 2, 3, 4] (adiciona no fim)
numeros.insert(0, 0) # [0, 1, 2, 3, 4] (insere na posição 0)
numeros.remove(2) # [0, 1, 3, 4] (remove o valor 2)
numeros.pop() # remove e retorna o último
numeros.pop(0) # remove e retorna o índice 0
len(numeros) # tamanho da lista
sum(inflacoes) # soma
```

```
max(inflacoes)          # maior valor
min(inflacoes)          # menor valor
sorted(inflacoes)       # lista ordenada (crescente)
sorted(inflacoes, reverse=True) # decrescente
```

8. Dicionários

Dicionário armazena pares **chave: valor**. É como uma tabela com índices personalizados.

```
# Criando
pib = {
    "Brasil": 2.2,
    "Argentina": -1.8,
    "Chile": 3.1,
    "Colômbia": 2.8
}

# Acessando
print(pib["Brasil"])      # 2.2
print(pib.get("Brasil"))  # 2.2 (seguro – não dá erro se não existir)
print(pib.get("Uruguai", 0.0)) # 0.0 (valor padrão)

# Adicionar/modificar
pib["Peru"] = 3.5
pib["Brasil"] = 2.5 # atualiza

# Métodos
pib.keys()    # todas as chaves
pib.values()  # todos os valores
pib.items()   # pares (chave, valor)

for pais, valor in pib.items():
    print(f"{pais}: {valor}%")
```

Quando usar lista vs dicionário:

Lista	Dicionário
[0.5, 0.7, 0.88]	{"jan": 0.5, "fev": 0.7}
Acesso por índice (0, 1, 2...)	Acesso por chave ("jan", "fev"...)
Dados sequenciais	Dados nomeados

9. Condicionais (if/elif/else)

```
ipca = 0.88

if ipca > 1.0:
    print("Inflação alta")
elif ipca > 0.5:
    print("Inflação moderada")
elif ipca > 0.2:
    print("Inflação controlada")
else:
    print("Deflação")
```

Operador ternário (if em uma linha):

```
status = "Alta" if ipca > 0.5 else "Baixa"
```

Múltiplas condições:

```
pib = -0.5
desemprego = 12.0

if pib < 0 and desemprego > 10:
    print("País em recessão com alto desemprego")
elif pib < 0 or desemprego > 10:
    print("Um dos indicadores está negativo")
```

10. Loops

10.1. for — percorrer sequências

```
# Percorrer lista
inflacoes = [0.50, 0.70, 0.88, 0.67]
for valor in inflacoes:
    print(f"IPCA: {valor}%")

# Percorrer com índice
for i, valor in enumerate(inflacoes):
    print(f"Mês {i+1}: {valor}%")

# Percorrer dicionário
pib = {"Brasil": 2.2, "Chile": 3.1}
for pais, valor in pib.items():
    print(f"{pais}: {valor}%")

# range() — repetir N vezes
for i in range(5):
    print(i) # 0, 1, 2, 3, 4

for i in range(1, 13):
    print(f"Mês {i}") # 1 a 12
```

10.2. while — repetir enquanto condição for verdadeira

```
saldo = 1000
taxa = 0.01
anos = 0

while saldo < 2000:
    saldo *= (1 + taxa)
    anos += 1

print(f"Leva {anos} anos para dobrar")

# CUIDADO: loop infinito! Sempre garanta que a condição
# vai se tornar False em algum momento
```

10.3. break e continue

```
# break — para o loop
for valor in inflacoes:
    if valor > 0.8:
        print("Alta detectada, parando")
        break
```

```

    print(valor)

# continue – pula para o próximo
for valor in inflacoes:
    if valor < 0.3:
        continue # pula valores baixos
    print(valor)

```

11. Funções

Função é um bloco de código reutilizável.

```

def nome_da_funcao(parametro1, parametro2):
    """Docstring: explica o que a função faz"""
    resultado = parametro1 + parametro2
    return resultado

```

11.1. Função sem parâmetro

```

def saudacao():
    print("Bem-vindo ao analisador econômico")

```

```
saudacao()
```

11.2. Função com parâmetros e retorno

```

def calcular_cambio_real(pib, cambio):
    """Calcula taxa de câmbio real simplificada"""
    return cambio * (1 + pib / 100)

```

```

cambio_real = calcular_cambio_real(3.2, 5.45)
print(f"Câmbio real ajustado: {cambio_real:.2f}")

```

11.3. Vários parâmetros

```

def taxa_juros_real(selic, ipca_esperado):
    """Calcula taxa de juros real (Fischer simplificado)"""
    return selic - ipca_esperado

```

```

juro_real = taxa_juros_real(10.5, 4.5)
print(f"Juro real: {juro_real:.1f}%")

```

11.4. Parâmetros com valor padrão

```

def pib_per_capita(pib_total, populacao, unidade="mil"):
    """Calcula PIB per capita com unidade configurável"""
    resultado = pib_total / populacao
    if unidade == "mil":
        return resultado / 1000
    elif unidade == "milhao":
        return resultado / 1_000_000
    return resultado

print(pib_per_capita(2_000_000_000_000, 214_000_000))
print(pib_per_capita(2_000_000_000_000, 214_000_000, unidade="milhao"))

```

11.5. Função que retorna múltiplos valores

```

def estatisticas_basicas(dados):
    """Retorna média, mínimo e máximo de uma lista"""
    media = sum(dados) / len(dados)

```

```
minimo = min(dados)
maximo = max(dados)
return media, minimo, maximo # retorna uma tupla

inf = [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]
media, min_inf, max_inf = estatisticas_basicas(inf)
print(f"Média: {media:.2f}, Mín: {min_inf:.2f}, Máx: {max_inf:.2f}")
```

12. List Comprehension (criação rápida de listas)

É uma forma concisa de criar listas. Muito usada em pandas.

```
# Sintaxe: [expressao for item in lista if condicao]

# Quadruplicar cada valor
inf = [0.5, 0.7, 0.88, 0.67]
dobro = [x * 2 for x in inf] # [1.0, 1.4, 1.76, 1.34]

# Filtrar
altos = [x for x in inf if x > 0.6] # [0.7, 0.88, 0.67]

# Aplicar função
def taxar(x):
    return x * 1.1

taxado = [taxar(x) for x in inf]

# Equivalente sem list comprehension:
altos = []
for x in inf:
    if x > 0.6:
        altos.append(x)
```

13. Erros comuns (e como ler o erro)

```
# NameError – variável não definida
print(valor_inexistente)

# TypeError – operação com tipo errado
"IPCA: " + 0.16 # erro! não pode somar str + float
"IPCA: " + str(0.16) # correto

# IndexError – índice fora da lista
lista = [1, 2]
lista[5] # erro! só existe índice 0 e 1

# KeyError – chave que não existe no dicionário
d = {"a": 1}
d["b"] # erro!

# ZeroDivisionError – divisão por zero
10 / 0
```

Toda vez que der erro, leia a ÚLTIMA linha primeiro. Ela diz o tipo do erro. As linhas acima mostram onde no seu código.

14. Exercícios

Tente fazer cada um no seu editor antes de olhar a solução.

Exercício 1

Crie uma função `inflacao_acumulada` que recebe uma lista de valores mensais e calcula a inflação acumulada no período (multiplique 1 + cada taxa).

Solução

```
def inflacao_acumulada(taxas):
    acumulado = 1.0
    for taxa in taxas:
        acumulado *= (1 + taxa / 100)
    return (acumulado - 1) * 100

taxas = [0.50, 0.70, 0.88, 0.67, 0.58, 0.16]
print(f"Acumulado: {inflacao_acumulada(taxas):.2f}%")
```

Exercício 2

Dado um dicionário com PIB de países, retorne o país com maior PIB e o com menor.

```
pibs = {"Brasil": 2.2, "Argentina": -1.8, "Chile": 3.1, "Peru": 2.5, "México": 1.9}
```

Solução

```
def maior_menor_pib(dados):
    maior_pais = max(dados, key=dados.get)
    menor_pais = min(dados, key=dados.get)
    return maior_pais, dados[maior_pais], menor_pais, dados[menor_pais]

maior_p, v_maior, menor_p, v_menor = maior_menor_pib(pibs)
print(f"Maior: {maior_p} ({v_maior}%")
print(f"Menor: {menor_p} ({v_menor}%")
```

Exercício 3

Simule juros compostos: comece com R\$ 1000, taxa de 1% ao mês, por 12 meses. Mostre o saldo a cada mês.

Solução

```
saldo = 1000
taxa = 0.01

for mes in range(1, 13):
    saldo *= (1 + taxa)
    print(f"Mês {mes:2d}: R$ {saldo:.2f}")
```

Exercício 4

Crie uma função que recebe uma lista de números e retorna uma nova lista apenas com valores acima da média.

Solução

```
def acima_da_media(dados):
    media = sum(dados) / len(dados)
    return [x for x in dados if x > media]

inf = [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]
print(acima_da_media(inf))
```

15. Resumo — o que você precisa lembrar

Conceito	Sintaxe
Print	<code>print(f"texto {var}")</code>
Lista	<code>lista = [1, 2, 3]</code>
Dicionário	<code>d = {"chave": valor}</code>
If/else	<code>if x > 0: / else:</code>
Loop	<code>for item in lista:</code>
Função	<code>def nome(x): return x</code>
List comp	<code>[x*2 for x in lista if x > 0]</code>
Comentário	<code># isso é comentário</code>

Próxima apostila

Quando terminar os exercícios acima, me chame para a **Apostila 2 — pandas para Economia**. Lá você vai aprender a abrir tabelas, filtrar dados, agrupar, juntar bases e trabalhar com datas — 90% do que um economista faz no dia a dia.